

---

# ICPC Algorithm Library

---

# 目录

|          |                    |          |
|----------|--------------------|----------|
| <b>1</b> | <b>Basic</b>       | <b>1</b> |
| 1.1      | test . . . . .     | 1        |
| 1.2      | pai . . . . .      | 1        |
| 1.3      | run . . . . .      | 1        |
| <b>2</b> | <b>Geometry</b>    | <b>1</b> |
| 2.1      | vector . . . . .   | 1        |
| 2.2      | seg_poly . . . . . | 1        |

# 1 Basic

## 1.1 test

```
export fs="-fsanitize=address,undefined"
ulimit -s 1024000
ulimit -v 1024000
mk() { g++ -o $1 $1.cpp -O2; }
```

## 1.2 pai

```
pai() {
    mk a; mk bf; mk gen
    while true; do
        ./gen > 1.in
        ./bf < 1.in > 1.ans
        ./a < 1.in > 1.out
        if diff -Zq 1.out 1.ans; then echo ac; else echo wa; break; fi
    done
}
```

## 1.3 run

```
shopt -s nullglob # 没有匹配到任何文件时，让它返回空
ulimit -s 1024000
g++ $1.cpp -o $1 -std=c++20 -O2 -fsanitize=address,undefined || exit
for f in "$2"/*.in; do
    x=${f%.in}
    \time -f "$x: %es %MKB" ./$1 < $x.in > $1.out
    diff -Zq $1.out $x.ans
done
```

# 2 Geometry

## 2.1 vector

```
using db = long double;
struct p2 {
    db x, y;
    db len() { return hypot(x, y); }
    p2 operator+(p2 b) { return {x + b.x, y + b.y}; }
    p2 operator-(p2 b) { return {x - b.x, y - b.y}; }
    p2 operator*(db k) { return {x * k, y * k}; }
    p2 operator/(db k) { return {x / k, y / k}; }
    db operator*(p2 b) { return x * b.x + y * b.y; } // 点积
    db operator%(p2 b) { return x * b.y - y * b.x; } // 叉积
};
```

```
using ps = vector<p2>;
```

## 2.2 seg\_poly

```
// 点到直线距离
db tol(p2 p, p2 a, p2 b) { return abs((b - a) % (p - a)) / (b - a).len(); }
```